



Aula de Introdução à Programação

Portugol Studio • Do “Olá, mundo!” às primeiras decisões

Uma aula prática para começar a programar com entrada, saída, variáveis, operadores, condições e repetições.

Roteiro da aula

A sequência foi organizada para sair do mais simples e chegar ao controle de fluxo.

1. Olá, mundo!

Executar o primeiro programa e entender a estrutura básica.

2. Entrada e saída

Inserir uma informação e fazer o programa devolver uma resposta.

3. Variáveis

Guardar dados como nome, idade, nota e resultado lógico.

4. Operadores

Fazer cálculos e comparar valores.

5. Condições

Tomar decisões com se, senao e escolha/caso.

6. Laços

Repetir tarefas.

A aula alterna explicação curta, demonstração e atividade rápida.

Cada exemplo possui código e saída esperada.

A ideia é construir confiança antes de aumentar a complexidade.

O que é programar?

Programar é escrever instruções para resolver uma tarefa de forma lógica.

Problema

Algo que precisa ser resolvido. Ex.: calcular média, validar idade, cadastrar aluno.

Algoritmo

Sequência ordenada de passos para resolver o problema.

Programa

Algoritmo escrito em uma linguagem que o computador interpreta.

Fluxo geral

Problema → Análise → Algoritmo → Programa → Resultado

Preparando o Portugol Studio

Antes de programar, o aluno precisa abrir a ferramenta corretamente.

Baixar

Acessar o link ou arquivo disponibilizado pelo professor.

Localizar

Encontrar o arquivo na pasta Downloads.

Extrair

Clicar com o botão direito e escolher "Extrair tudo".

Abrir pasta

Entrar na pasta extraída e manter os arquivos juntos.

Executar

Dar dois cliques no aplicativo do Portugol.

Código

```
programa
{
    funcao inicio()
    {
        escreva("Pronto para começar!")
    }
}
```

Saída esperada

Pronto para começar!

Primeiro programa: Olá, mundo!

O objetivo é executar algo simples e ver uma resposta na tela.

Código

```
programa
{
    funcao inicio()
    {
        escreva("Olá, mundo!")
    }
}
```

Saída esperada

Olá, mundo!

O que observar

A saída é exatamente o texto que está entre aspas no comando escreva.

Digite o código sem copiar e colar.
Execute e observe a saída.
Altere a frase e execute novamente.

Como o Portugol lê esse código?

A estrutura básica é repetida em quase todos os exemplos.

programa

Indica a estrutura principal do arquivo.

funcao inicio()

Ponto inicial da execução.

{ }

Delimitam blocos de comandos.

escreva()

Exibe texto ou valores na tela.

Código

```
programa
{
    funcao inicio()
    {
        escreva("Mensagem")
    }
}
```

Mostrando várias linhas na saída

Use vários comandos escreva ou o caractere de nova linha.

Código

```
programa
{
    funcao inicio()
    {
        escreval("Nome: Ana")
        escreval("Curso: Programação")
        escreva("Ferramenta: Portugol")
    }
}
```

Saída esperada

```
Nome: Ana
Curso: Programação
Ferramenta: Portugol
```

Dica

escreval significa quebra de linha: o próximo texto aparece abaixo.

Agora vamos inserir informação

A saída fixa é útil, mas programas reais precisam receber dados.

Entrada

O usuário digita uma informação.

Processamento

O programa guarda, calcula ou compara.

Saída

O programa devolve uma resposta.

Código

```
escreva("Digite seu nome: ")  
leia(nome)  
escreva("Olá, ", nome)
```

Saída esperada

```
Digite seu nome: Beatriz  
Olá, Beatriz
```

Variável: onde o dado fica guardado

Para ler uma informação, precisamos preparar um espaço para armazená-la.

Ideia

Variável é como uma caixa com nome.

Exemplo

nome guarda um texto digitado pelo usuário.

Importante

O tipo da variável indica que dado ela aceita.

Código

```
cadeia nome

escreva("Digite seu nome: ")
leia(nome)

escreva("Olá, ", nome)
```

Saída esperada

```
Digite seu nome: Carlos
Olá, Carlos
```

Programa completo: ler e devolver uma informação

Este é o primeiro programa interativo da aula.

Código

```
inicio()

  cadeia nome

  escreva("Digite seu nome: ")
  leia(nome)

  escreva("Olá, ", nome, "! Seja bem-vindo.")
```

Saída esperada

```
Digite seu nome: Marina
Olá, Marina! Seja bem-vindo.
```

Atividade rápida

Troque nome por cidade e faça o programa responder: "Você mora em ..."

Tipos de dados mais usados

Cada tipo representa uma categoria de informação.

inteiro

Números sem casas decimais. Ex.: 18, 2026.

real

Números com casas decimais. Ex.: 7.5, 1500.90.

cadeia

Textos. Ex.: "Ana", "Londrina".

caracter

Um único símbolo. Ex.: 'A'.

logico

verdadeiro ou falso.

Código

```
inteiro idade = 18
real media = 8.5
caracter nome = "Ana"
caracter turma = 'B'
logico aprovado = verdadeiro
```

Saída esperada

```
Ana - idade 18
Média: 8.5
Aprovado: verdadeiro
```

Nomes de variáveis: boas práticas

O nome deve explicar o que está guardado.

Evite nomes genéricos

x, a, coisa, dado, valor1 sem contexto.

Prefira nomes claros

idade, media_final, total_compra, nome_aluno.

Cuidado com espaços

Use sublinhado: nome_aluno, valor_total.

Código

```
// pouco claro
real x

// mais claro
real media_final
real valor_total
cadeia nome_aluno
```

Saída esperada

O programa fica mais legível
e mais fácil de corrigir.

Inserindo números e calculando

Depois de ler dados, podemos processar as informações.

Código

```
inicio()

    real nota1, nota2, media

    escreva("Nota 1: ")
    leia(nota1)

    escreva("Nota 2: ")
    leia(nota2)

    media = (nota1 + nota2) / 2.0

    escreva("Média: ", media)
```

Saída esperada

```
Nota 1: 8
Nota 2: 7
Média: 7.5
```

O que mudou?

Agora existe processamento: o programa calcula antes de responder.

Operadores aritméticos

São usados para realizar cálculos.

+

Soma

-

Subtração

*

Multiplicação

/

Divisão

%

Resto da divisão

Código

```
real preco = 100.0
real desconto = 15.0
real total

total = preco - desconto

escreva("Total: ", total)
```

Saída esperada

Total: 85.0

Exemplo: conversão de temperatura

Um cálculo simples mostra entrada, processamento e saída.

Código

```
inicio()

    real celsius, fahrenheit

    escreva("Temperatura em Celsius: ")
    leia(celsius)

    fahrenheit = (celsius * 9.0 / 5.0) + 32.0

    escreva("Fahrenheit: ", fahrenheit)
```

Saída esperada

```
Temperatura em Celsius: 25
Fahrenheit: 77.0
```

Atividade rápida

Modifique o exemplo para converter metros em centímetros.

Operadores relacionais

Eles comparam valores e produzem verdadeiro ou falso.

>

maior que

<

menor que

>=

maior ou igual

<=

menor ou igual

==

igual

!=

diferente

Código

```
real media = 7.5  
  
escreva(media >= 7.0)  
escreva("\n")  
escreva(media < 5.0)
```

Saída esperada

```
verdadeiro  
falso
```

Operadores lógicos: e, ou, nao

Eles combinam mais de uma condição.

e

Verdadeiro apenas quando todas as condições são verdadeiras.

ou

Verdadeiro quando pelo menos uma condição é verdadeira.

nao

Inverte o resultado lógico.

Código

```
real media = 8.0
real frequencia = 80.0

logico aprovado
aprovado = media >= 7.0 e frequencia >= 75.0

escreva(aprovado)
```

Saída esperada

verdadeiro



Tópico: estruturas de controle

Agora o programa começa a escolher caminhos.

Condição simples: se

Executa um bloco apenas quando a condição é verdadeira.

Código

```
idade: inteiro

escreva("Idade: ")
leia(idade)

se (idade >= 18) entao
escreva("Entrada permitida.")
senao
escreva("Entrada não permitida.") fimse
```

Saída esperada

```
Idade: 20
Maior de idade
```

Importante

Se a idade for menor que 18, será escrito nesse exemplo “Entrada não permitida”

Condição composta: se...senao

Escolhe entre dois caminhos possíveis.

Código

```
se (media >= 7.0) entao  
    escreva("Aprovado")  
  
senao  
    escreva("Reprovado")  
fimse
```

Saída esperada

Saída com média 8.0:
Aprovado

Saída com média 4.5:
Reprovado

Como explicar

O se testa a condição. Se ela for falsa, o programa pula para o senao.

Condição encadeada: mais de duas situações

Ideal para classificar aprovado, recuperação e reprovado.

Código

```
se (media >= 7.0) entao
  escreva("Aprovado")
senao
  se (media >= 5.0) entao
    escreva("Recuperação")
  senao
    escreva("Reprovado")
  fimse
fimse
```

Saída esperada

```
média = 8.0 → Aprovado
média = 6.0 → Recuperação
média = 3.5 → Reprovado
```

A ordem importa

O programa testa de cima para baixo e para no primeiro caso verdadeiro.

Múltiplas opções: escolha/caso

Útil para menus e seleção de alternativas.

Código

```
escolha (opcao)
  caso 1
    escreva("Cadastrar")
  caso 2
    escreva("Consultar")
  outrocaso
    escreva("Opção inválida")
fimescolha
```

Saída esperada

```
1 - Cadastrar
2 - Consultar
Escolha: 2
Consultar
```

Quando usar

Quando o usuário escolhe uma opção específica, como 1, 2 ou 3.



Tópico: estruturas de repetição

Laços permitem executar a mesma lógica várias vezes.

Por que repetir?

Sem repetição, seria necessário copiar o mesmo código várias vezes.

Exemplo

Cadastrar notas de vários alunos.

Sem laço

Copiar o mesmo código para aluno 1, aluno 2, aluno 3...

Com laço

Escrever uma vez e repetir até terminar.

Código

```
para i de 1 ate 3 faca  
    escreval("Aluno ", i)  
fimpara
```

Saída esperada

```
Aluno 1  
Aluno 2  
Aluno 3
```

Repetição para

Use quando a quantidade de repetições é conhecida.

Código

```
Var i: inteiro

Inicio // Seção de Comandos, procedimento, funções, operadores, etc...

    para i de 1 ate 5 faca
        escreval("Número ", i)
    fimpara

Fimalgoritmo
```

Saída esperada

```
Número 1
Número 2
Número 3
Número 4
Número 5
```

Partes do para

início → limite → incremento

Repetição enquanto

Use quando a repetição depende de uma condição.

Código

```
Var opcao: inteiro

Inicio // Seção de Comandos, procedimento, funções, operadores, etc...

opcao <- 0
enquanto (opcao <> 3) faca
  escreval("1 - Cadastrar")
  escreval("2 - Consultar")
  escreval("3 - Sair")
  leia(opcao)
fimenquanto

Fimalgoritmo
```

Saída esperada

O menu continua aparecendo
até o usuário escolher 3.

Cuidado

Se a variável da condição nunca mudar, o laço pode ficar infinito.

Repetição faça...enquanto

Executa pelo menos uma vez e depois testa a condição.

Código

```
Var  
nota: real  
  
Inicio // Seção de Comandos, procedimento, funções, operadores, etc...  
  
  repita  
    escreva("Digite uma nota entre 0 e 10: ")  
    leia(nota)  
    ate (nota >= 0.0) e (nota <= 10.0)  
    escreva("Nota válida: ", nota)  
  
Fimalgoritmo
```

Saída esperada

```
Digite uma nota entre 0 e 10: 15  
Digite uma nota entre 0 e 10: 8  
Nota válida: 8.0
```

Quando usar

Validação de entrada: senha, nota, idade, opção de menu.

Contador e acumulador

Dois padrões muito comuns em programas com repetição.

Contador

Conta quantas vezes algo aconteceu. Ex.: quantidade de aprovados.

Acumulador

Soma valores ao longo do laço. Ex.: soma das médias da turma.

Código

```
inteiro aprovados = 0
real soma_medias = 0.0

se (media >= 7.0) entao
  aprovados <- aprovados + 1
fimse
soma_medias <- soma_medias + media
```

Saída esperada

```
aprovados aumenta de 1 em 1
soma_medias recebe uma nova média
```



Tópico: funções e procedimentos

Blocos reutilizáveis deixam o código mais organizado.

Função: reaproveitar um cálculo

Funções podem receber valores e devolver um resultado.

Código

```
Var
media_final: real

funcao calcular_media(n1, n2: real): real
Inicio // O retorno é feito atribuindo o valor ao nome da função
calcular_media <- (n1 + n2) / 2
fimfuncao
I
media_final <- calcular_media(8.0, 6.0) escreva("Média: ", media_final)
Fimalgoritmo
```

Saída esperada

Média: 7.0

Vantagem

Se a fórmula mudar, alteramos a função em um único lugar.

Procedimento: reaproveitar uma sequência de comandos

Nem todo bloco precisa devolver um valor.

Código

```
funcao mostrar_linha()
{
    escreval("-----")
}

funcao inicio()
{
    mostrar_linha()
    escreval("MENU PRINCIPAL")
    mostrar_linha()
}
```

Saída esperada

```
-----
MENU PRINCIPAL
-----
```

Quando usar

Cabeçalho, menu, linha divisória, mensagem padrão ou validações repetidas.



Projeto prático da aula

Sistema simples de controle de notas e frequência.

Como os tópicos entram no projeto

Cada conteúdo da aula resolve uma parte do sistema.

Variáveis

Guardar nome, notas, frequência e média.

Operadores

Calcular média e comparar limites.

Condições

Classificar aprovado, recuperação ou reprovado.

Repetições

Cadastrar vários estudantes.

Funções

Separar cálculo, validação e classificação.

Código

```
Entrada: nome, notas e frequência  
Processamento: média + classificação  
Saída: situação do aluno + resumo da turma
```

Saída esperada

```
Ana → Média 8.5 → Aprovada  
Bruno → Média 5.5 → Recuperação  
Resumo: 1 aprovado, 1 recuperação
```

Mini-exemplo do projeto

Uma versão inicial para um estudante.

Código

```
programa
{
    funcao inicio()
    {
        cadeia nome
        real nota1, nota2, frequencia, media

        escreva("Nome: ")
        leia(nome)
        escreva("Nota 1: ")
        leia(nota1)
        escreva("Nota 2: ")
        leia(nota2)
        escreva("Frequência: ")
        leia(frequencia)

        media = (nota1 + nota2) / 2.0

        se (frequencia < 75.0) { escreva("Reprovado por frequência") }
        senao se (media >= 7.0) { escreva("Aprovado") }
        senao se (media >= 5.0) { escreva("Recuperação") }
        senao { escreva("Reprovado") }
    }
}
```

Saída esperada

```
Nome: Ana
Nota 1: 8
Nota 2: 7
Frequência: 90
Aprovado
```

Evolução

Depois, repetimos esse cadastro para vários estudantes.

Atividade guiada em sala

Construir aos poucos, testando cada etapa.

Parte 1

Criar Olá, mundo e alterar a mensagem.

Parte 2

Ler nome e devolver uma saudação personalizada.

Parte 3

Ler duas notas e calcular a média.

Parte 4

Usar se/senao para classificar a média.

Parte 5

Criar repetição para vários estudantes.

Parte 6

Adicionar resumo com contadores.

Produto esperado

Um programa simples capaz de ler dados, calcular média, classificar o aluno e apresentar resultado.

Erros comuns no início

Errar faz parte, mas é importante aprender a interpretar a mensagem.

Esquecer aspas

Textos precisam ficar entre aspas.

Esquecer variável

Antes de usar `leia(nome)`, declare cadeia nome.

Laço infinito

A condição nunca muda e o programa não para.

Código

```
// erro comum  
escreva(Olá)  
  
// correto  
escreva("Olá")
```

Saída esperada

Revise linha, chaves, aspas,
variáveis e condições.

Resumo da aula

O caminho do primeiro código até o controle de fluxo.

Começamos

Olá, mundo e estrutura básica.

Inserimos dados

leia, variáveis e saída personalizada.

Calculamos

operadores aritméticos e fórmulas.

Decidimos

se, senao, senao se e escolha/caso.

Repetimos

para, enquanto e faca-enquanto.

Sequência lógica da programação

Entrada → Processamento → Decisão/Repetição → Saída

Próximo passo

Praticar com exercícios e evoluir o projeto de notas e frequência.